LAB 7

Fuzzing for Input Validation Bugs

Using Python and Hypothesis

Details	Information
Name	SATWIK INCHAGERI
USN	PES1UG24CS618
Subject / section	SE SECTION K
Date	13-11-2026

1. Aim

The aim of this lab was to use fuzz testing to find input validation problems in the given Python program. I used the Hypothesis library to generate different inputs, checked which inputs caused errors, fixed the functions and ran the tests again.

2. Software used

Software	Version
Python	3.14.7
pip	26.2.2
Hypothesis	6.170.15
pytest	9.2.0
Operating system	Windows 10.0.26300.9818

3. About the given program

The given processor.py file had three functions:

- parse_storage_quota() – takes a storage value such as 500MB or 2TB and converts it into GB.
- compute_compression_ratio() – calculates raw bytes divided by compressed bytes.
- lookup_bandwidth_tier_rate() – returns the rate for a storage tier such as HOT, COLD or ARCHIVE.

4. Steps followed

1. Installed Hypothesis and pytest using pip.
2. Downloaded processor.py and test_processor.py from the assigned GitHub repository.
3. Ran the original program with 500MB, 1024:256 and HOT_TIER.
4. Completed the three test functions in test_processor.py using Hypothesis.
5. Ran pytest on the original program and noted the failures.
6. Copied the original processor.py to processor_original.py as a backup.
7. Added input checks to the three functions in processor.py.
8. Ran the fixed program and tested invalid inputs.
9. Ran the same pytest command again to check whether the errors were fixed.

5. Hypothesis tests

I used @given(st.text() | st.none()) for each test. Hypothesis generated different strings, empty strings and None values. Each test called one function inside a try block. If the function raised an exception, the except block failed the test and printed the input that caused the problem.

The three test functions were:

- test_parse_storage_quota_no_crash
- test_compute_compression_ratio_safe

- test_lookup_bandwidth_tier_rate_safe

6. Results before fixing the program

The original program worked for the normal inputs. It displayed 0.50 GB for 500MB, 4.0 for 1024:256 and $0.08 for HOT_TIER. But the fuzz tests found errors when they tried unexpected inputs.

Function tested	Failures found	Inputs shown by Hypothesis
parse_storage_quota()	3	'000', empty string, None
compute_compression_ratio()	3	':', empty string, None
lookup_bandwidth_tier_rate()	2	empty string, None
Total	8	All three tests failed

The quota function failed because it did not check whether the input was a string or whether the number and unit were valid. The compression function failed when the input did not contain two valid numbers separated by a colon. The tier function failed for None and empty input, and could also raise a KeyError for an unknown tier.

The first pytest run ended with: 3 failed in 9.79s.

7. Changes made to processor.py

I made the following changes:

- For parse_storage_quota(), I checked the input type and used a regular expression to accept only a number followed by MB, GB or TB. Invalid values return None.
- For compute_compression_ratio(), I checked that the input is a string with exactly two parts. I also checked that the values are valid finite numbers and that the compressed size is greater than zero.
- For lookup_bandwidth_tier_rate(), I checked the input type and used dictionary get() instead of direct indexing. An unknown tier now returns None.

The main() function was also changed to display an invalid-input message when a function returns None. The original file was kept as processor_original.py.

8. Results after fixing the program

I first ran the fixed program with the same valid inputs. The outputs were unchanged.

Input	Output
500MB	Quota: 0.50 GB
1024:256	Compression Ratio: 4.0
HOT_TIER	Rate per GB: $0.08

I also tried invalid inputs directly. parse_storage_quota(None), compute_compression_ratio(':') and lookup_bandwidth_tier_rate('') each returned None instead of crashing.

Finally, I ran:

```
python -m pytest test_processor.py -v
```

The final output showed that all three tests passed in 2.18 seconds.

Test	Result
test_parse_storage_quota_no_crash	PASSED
test_compute_compression_ratio_safe	PASSED
test_lookup_bandwidth_tier_rate_safe	PASSED
Final result	3 passed in 2.18s

9. Conclusion

In this lab, I used Hypothesis to test the three functions with unexpected inputs. The tests found eight failure cases in the original program. I then added input validation and safe error handling to the code. After making these changes, I ran the same tests again and all three passed. This lab helped me understand how fuzz testing can find problems that may not appear when testing only normal inputs.

10. Files prepared

- processor.py – updated application.
- test_processor.py – Hypothesis test file.
- processor_original.py – backup of the original application.
- This report – screenshots and experiment results.

Screenshots

The screenshots below are from the actual commands run during the lab. They show the installation, downloaded files, original program, test code, failures and final successful test run.

Figure 1: Python version and installation of Hypothesis and pytest.

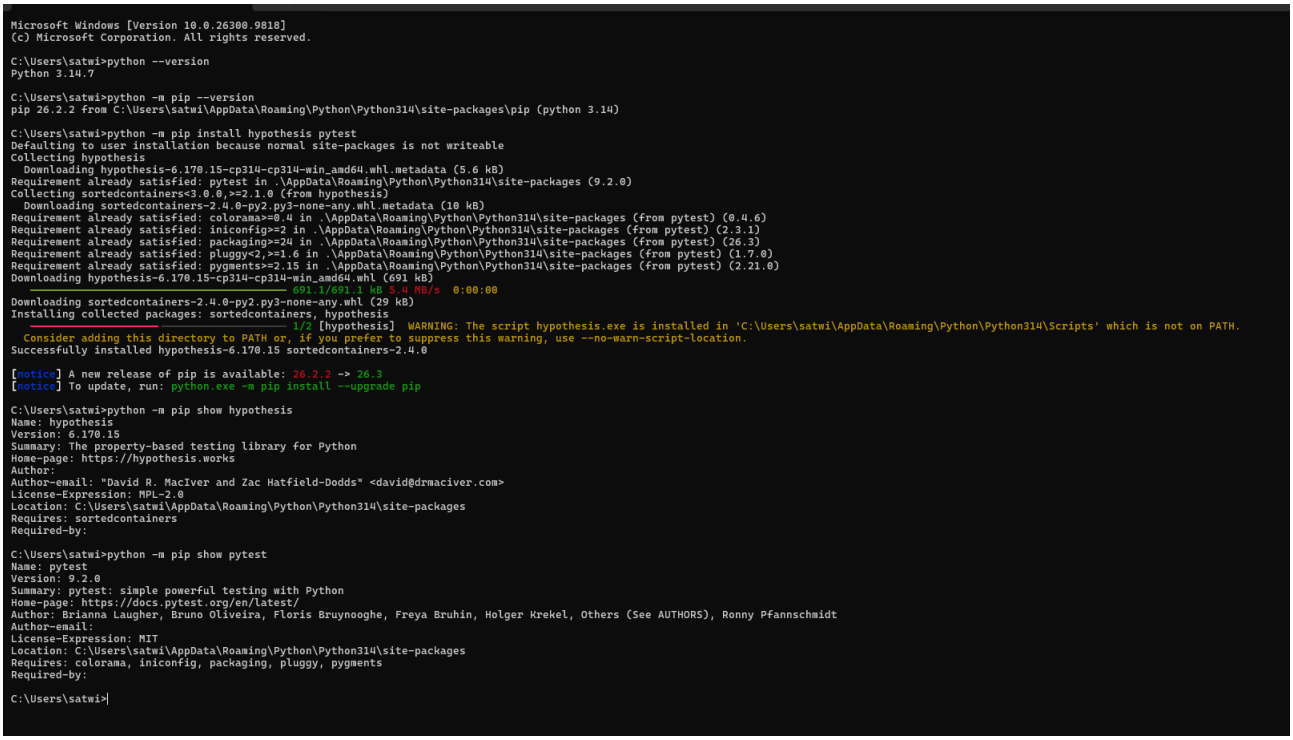


Figure 2: Downloading the two Lab 7 Python files.

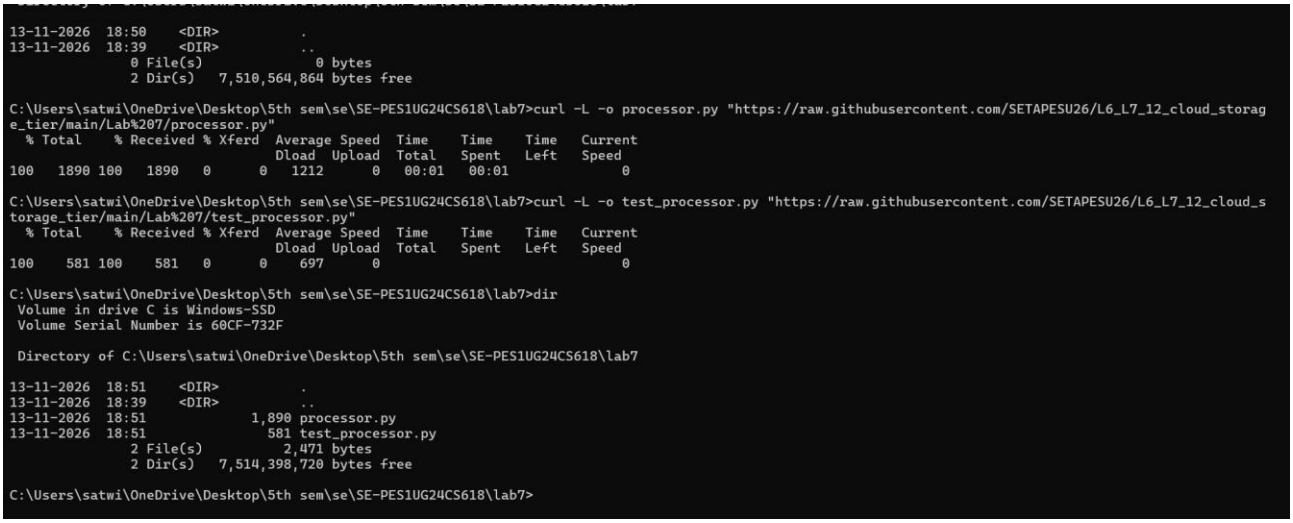


Figure 3: Running the original processor.py with valid inputs.

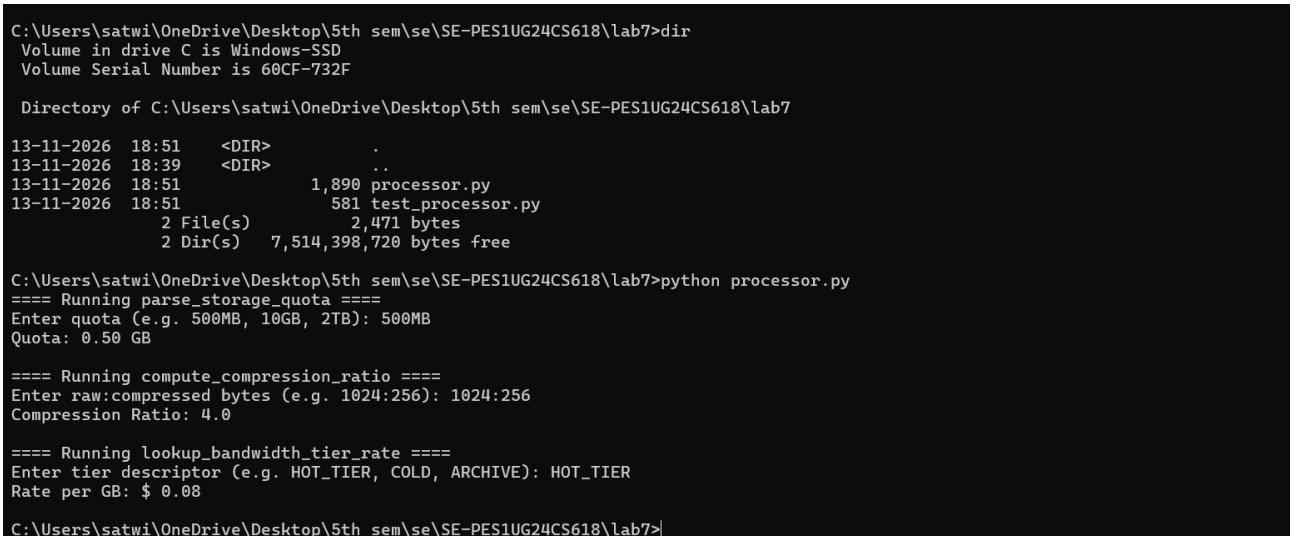


Figure 4: Completed Hypothesis tests in test_processor.py.

```
C:\Users\satwi\OneDrive\Desktop\5th sem\se\SE-PES1UG24CS618\lab7>notepad test_processor.py

C:\Users\satwi\OneDrive\Desktop\5th sem\se\SE-PES1UG24CS618\lab7>type test_processor.py

from hypothesis import given, strategies as st
from processor import (
    parse_storage_quota,
    compute_compression_ratio,
    lookup_bandwidth_tier_rate
)


@given(st.text() | st.none())
def test_parse_storage_quota_no_crash(s):
    try:
        parse_storage_quota(s)
    except Exception as e:
        assert False, f"parse_storage_quota crashed with input {s!r}: {e}"


@given(st.text() | st.none())
def test_compute_compression_ratio_safe(s):
    try:
        compute_compression_ratio(s)
    except Exception as e:
        assert False, f"compute_compression_ratio crashed with input {s!r}: {e}"


@given(st.text() | st.none())
def test_lookup_bandwidth_tier_rate_safe(s):
    try:
        lookup_bandwidth_tier_rate(s)
    except Exception as e:
        assert False, f"lookup_bandwidth_tier_rate crashed with input {s!r}: {e}"

C:\Users\satwi\OneDrive\Desktop\5th sem\se\SE-PES1UG24CS618\lab7>
```

Figure 5: Fuzz-testing failures in the quota parser and compression function.

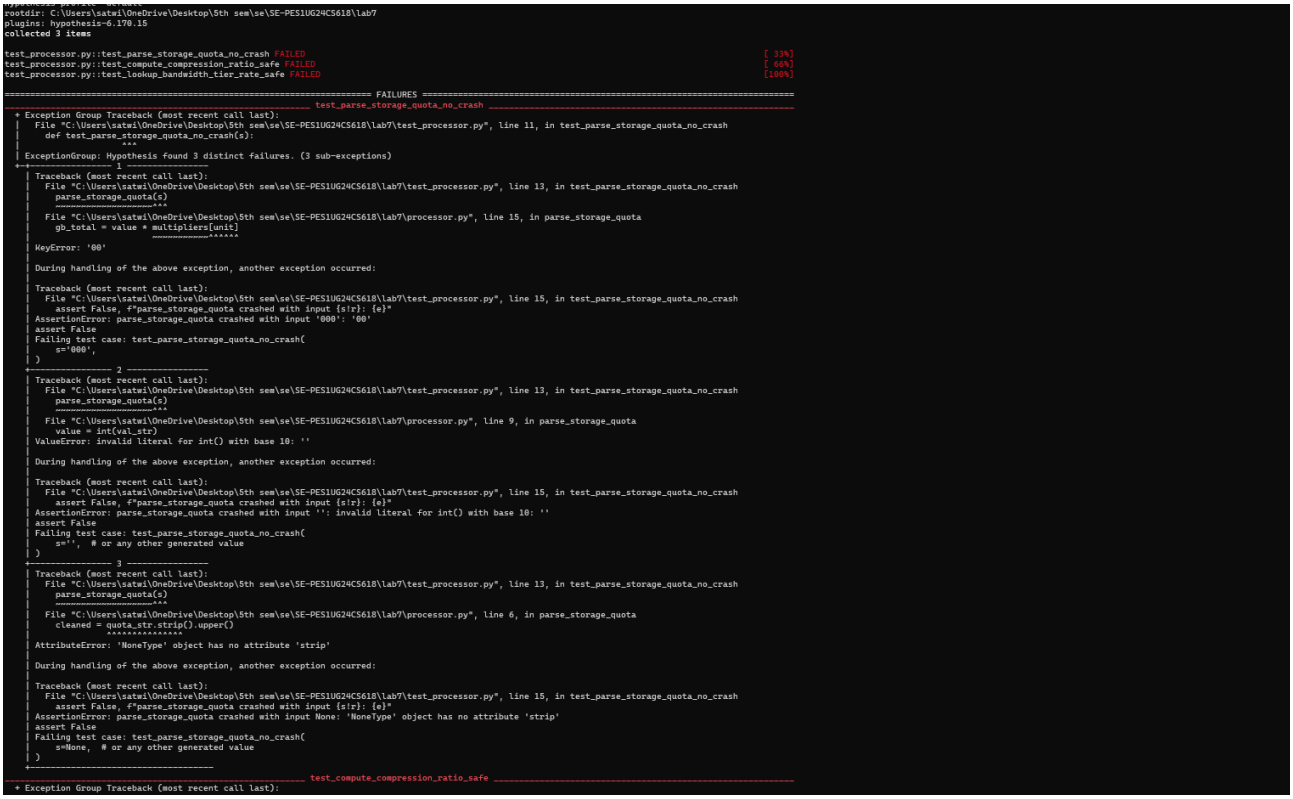


Figure 6: More fuzz-testing failure details.

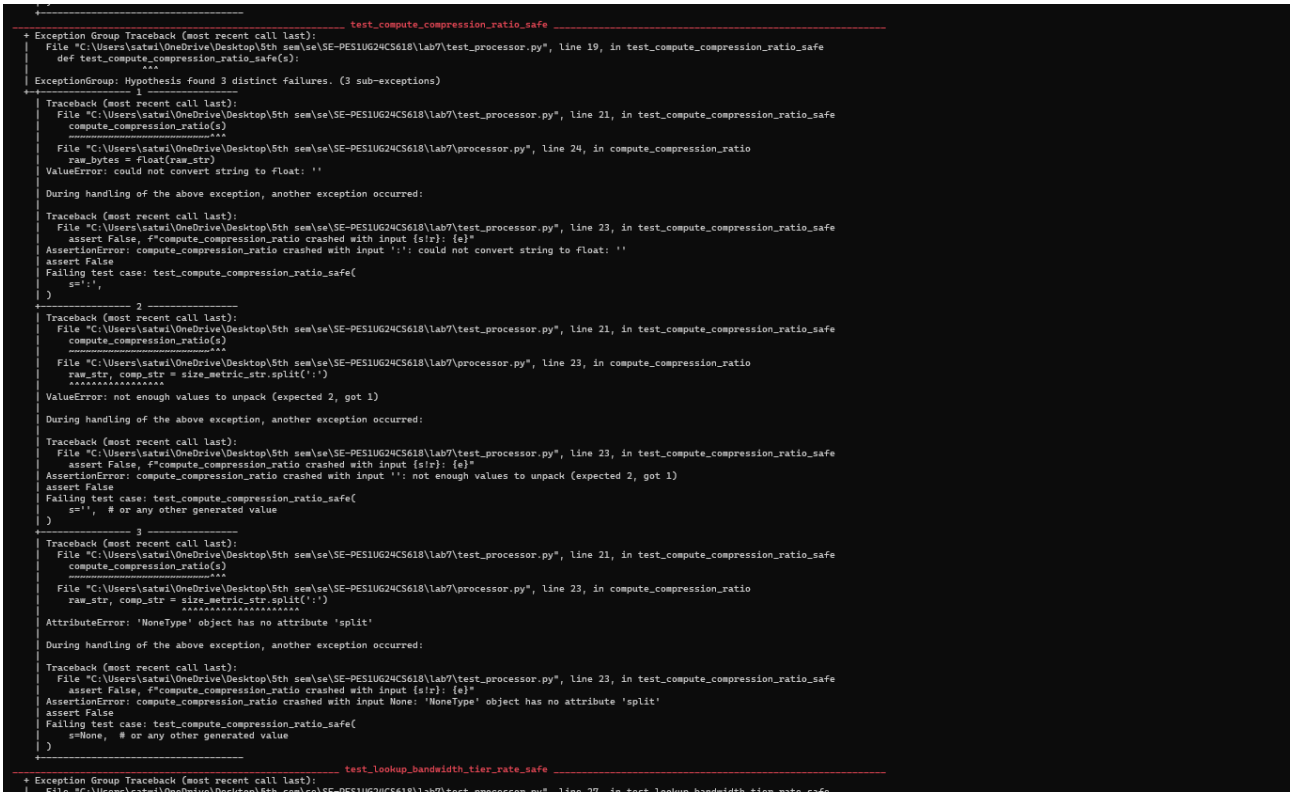


Figure 7: Final failure summary for the original program.

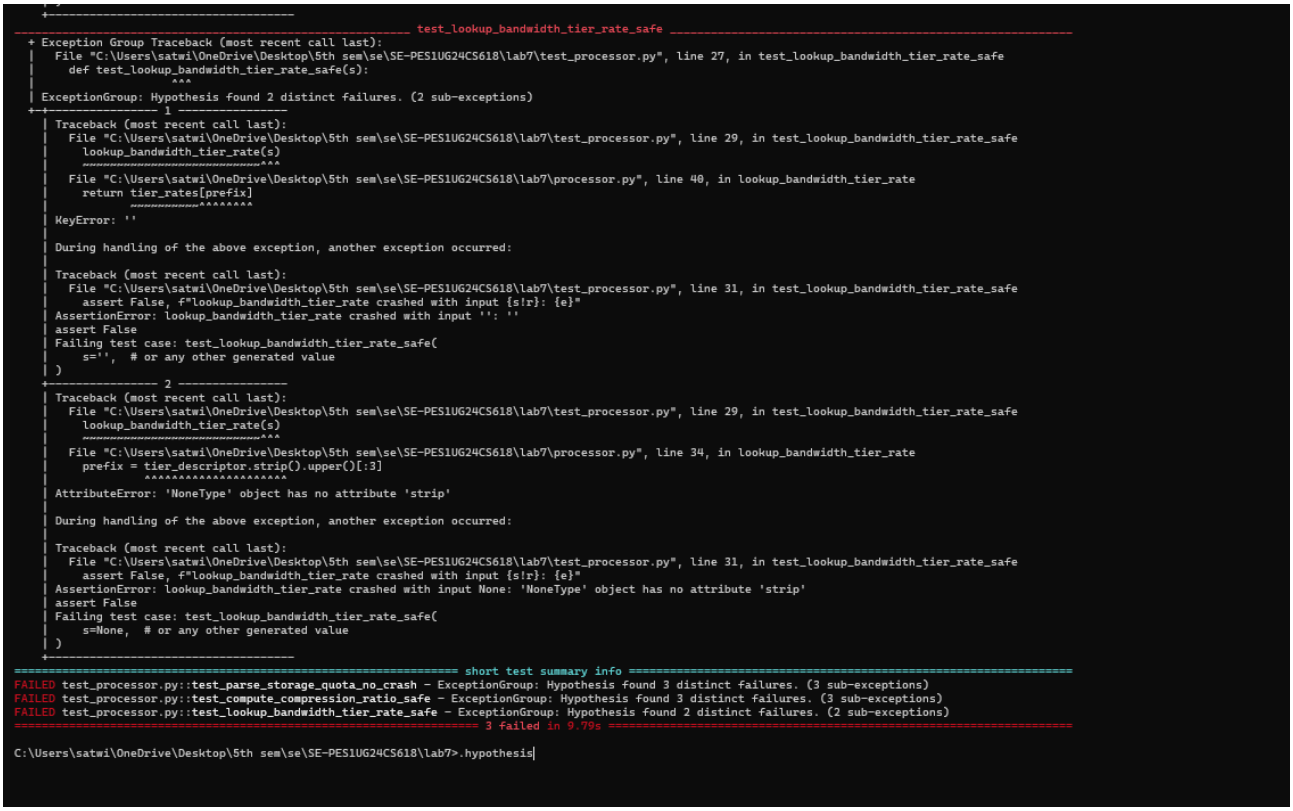


Figure 8: Fixed program output and invalid-input checks.

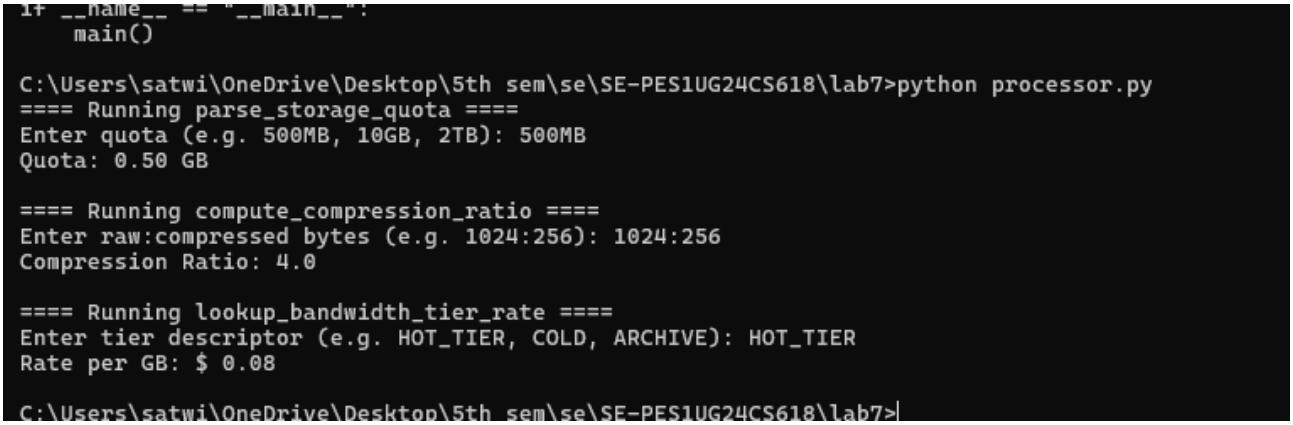


Figure 9: Final pytest output showing three passed tests.

